

LAPORAN PRAKTIKUM SISTEM OPERASI
DINING PHILOSOPHERS PROBLEM



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Disusun Oleh :
Subhan Maulana Andrianto 2410651014

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DAFTAR ISI

1. **Pendahuluan**
 - 1.1 Latar Belakang
 - 1.2 Tujuan Praktikum
 - 1.3 Dasar Teori
2. **Permasalahan Dining Philosophers**
 - 2.1 Deskripsi Masalah
 - 2.2 Kondisi Terjadinya Deadlock
 - 2.3 Analisis Empat Syarat Deadlock
3. **Implementasi Program**
 - 3.1 Kode Program
 - 3.2.1 Versi Deadlock (Demonstrasi)
 - 3.2.2 Versi Resource Ordering (Pencegahan Deadlock)
 - 3.2.3 Versi Limiting Philosophers (Pencegahan dengan Semaphore)
4. **Kesimpulan dan Saran**
 - 4.1 Kesimpulan
 - 4.2 Saran
5. **Daftar Pustaka**

PENDAHULUAN

1.1 Latar Belakang

Dalam sistem operasi, sinkronisasi merupakan hal penting untuk mengatur eksekusi proses yang berjalan secara bersamaan. Salah satu permasalahan klasik dalam sinkronisasi adalah Dining Philosophers Problem, yang menggambarkan permasalahan *deadlock* akibat sumber daya bersama (shared resources) seperti *fork* atau sumpit.

Permasalahan ini sering digunakan untuk mempelajari cara mencegah deadlock menggunakan teknik seperti mutual exclusion, semaphore, dan resource ordering.

1.2 Tujuan Praktikum

1. Memahami konsep sinkronisasi antar proses/thread.
2. Mengetahui kondisi terjadinya *deadlock* pada sistem paralel.
3. Mengimplementasikan algoritma pencegahan *deadlock* pada kasus Dining Philosophers.
4. Membandingkan hasil antara kondisi *deadlock* dan *deadlock prevention*.

1.3 Dasar Teori

- Deadlock adalah keadaan di mana dua atau lebih proses saling menunggu sumber daya yang dipegang oleh proses lain sehingga tidak ada proses yang bisa melanjutkan eksekusi.
- Empat syarat terjadinya deadlock (Coffman Conditions):
 1. *Mutual Exclusion*
 2. *Hold and Wait*
 3. *No Preemption*
 4. *Circular Wait*
- Semaphore dan Mutex digunakan untuk mengontrol akses terhadap sumber daya bersama agar tidak terjadi konflik.
- Dining Philosophers Problem digunakan sebagai model pembelajaran sinkronisasi dan pencegahan deadlock dengan cara:
 - Mengubah urutan pengambilan sumber daya (Resource Ordering).
 - Membatasi jumlah thread yang dapat mengakses sumber daya (Limiting Philosophers).

PERMASALAHAN DINING PHILOSOPHERS

2.1 Deskripsi Masalah

Terdapat lima filsuf yang duduk melingkar di meja makan, masing-masing memiliki satu sumpit di sisi kanan dan kiri. Setiap filsuf harus mengambil dua sumpit (kiri dan kanan) untuk dapat makan. Jika semua filsuf mengambil sumpit kiri secara bersamaan, maka tidak ada yang dapat mengambil sumpit kanan → **terjadi deadlock**.

2.2 Kondisi Terjadinya Deadlock

Deadlock terjadi ketika setiap filsuf:

1. Sudah memegang satu sumpit (misalnya sumpit kiri).
2. Menunggu sumpit kanan yang sedang dipegang oleh tetangga.
3. Tidak ada filsuf yang melepaskan sumpit.
Sehingga seluruh thread *terblokir* dan program berhenti tanpa ada aktivitas makan.

2.3 Analisis Empat Syarat Deadlock

No	Kondisi	Terpenuhi pada Dining Philosophers
1	Mutual Exclusion	Sumpit hanya bisa dipegang satu filsuf pada satu waktu.
2	Hold and Wait	Setiap filsuf memegang satu sumpit dan menunggu yang lain.
3	No Preemption	Sumpit tidak bisa direbut oleh filsuf lain.
4	Circular Wait	Setiap filsuf menunggu sumpit yang dipegang oleh tetangga (sirkular).

IMPLEMENTASI PROGRAM

3.2 Kode Program

3.2.1 Versi Deadlock (Demonstrasi)

Program menggunakan mutex untuk setiap sumpit, dan setiap filsuf selalu mengambil sumpit kiri dulu lalu kanan. Akibatnya, dapat terjadi deadlock saat semua filsuf menunggu sumpit kanan.

Input :

```
GNU nano 7.2 dining_philosophers.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_PHIL 5

pthread_mutex_t chopstick[NUM_PHIL];
pthread_t philosopher[NUM_PHIL];
int phil_id[NUM_PHIL];
sem_t room;

void* philosopher_deadlock(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % NUM_PHIL;
    while (1) {
        printf("🍲 Filsuf %d sedang berpikir...\n", id);
        sleep(1);
        printf("😋 Filsuf %d lapar dan mencoba mengambil sumpit kiri %d\n", id, left);
        pthread_mutex_lock(&chopstick[left]);
        printf("🍴 Filsuf %d memegang sumpit kiri %d\n", id, left);
        sleep(1);
        printf("😋 Filsuf %d mencoba mengambil sumpit kanan %d\n", id, right);
        pthread_mutex_lock(&chopstick[right]);
        printf("🍴 Filsuf %d sedang makan...\n", id);
        sleep(2);
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);
        printf("🍲 Filsuf %d kembali berpikir...\n", id);
    }
}

void* philosopher_ordering(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % NUM_PHIL;
    while (1) {
        printf("🍲 Filsuf %d berpikir...\n", id);
        sleep(1);
        printf("😋 Filsuf %d lapar...\n", id);
        if (id % 2 == 0) {
            pthread_mutex_lock(&chopstick[left]);
            pthread_mutex_lock(&chopstick[right]);
        } else {
            pthread_mutex_lock(&chopstick[right]);
            pthread_mutex_lock(&chopstick[left]);
        }
        printf("🍴 Filsuf %d makan (solusi A)\n", id);
        sleep(2);
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);
        printf("🍲 Filsuf %d kembali berpikir...\n", id);
    }
}

void* philosopher_limit(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % NUM_PHIL;
    while (1) {
        printf("🍲 Filsuf %d berpikir...\n", id);
        sleep(1);
    }
}
```

```

        printf("🍴 Filsuf %d lapar dan mencoba masuk ruang makan...\n", id);
        sem_wait(&room);
        pthread_mutex_lock(&chopstick[left]);
        pthread_mutex_lock(&chopstick[right]);
        printf("🍴 Filsuf %d makan (solusi B)\n", id);
        sleep(2);
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);
        sem_post(&room);
        printf("🍴 Filsuf %d selesai makan dan kembali berpikir...\n", id);
    }
}

int main() {
    for (int i = 0; i < NUM_PHIL; i++) {
        pthread_mutex_init(&chopstick[i], NULL);
        phil_id[i] = i;
    }
    sem_init(&room, 0, 4);

    int choice;
    printf("=== DINING PHILOSOPHERS PROBLEM ===\n");
    printf("Kondisi:\n");
    printf("- 5 filsuf duduk melingkar\n");
    printf("- 5 sumpit (1 di antara tiap filsuf)\n");
    printf("- Filsuf perlu 2 sumpit untuk makan\n");
    printf("- Filsuf berpikir → lapar → makan → berpikir (siklus)\n");
    printf("Pilih mode:\n");
    printf("1. Deadlock Demonstration\n");
    printf("2. Solution A - Resource Ordering (even/odd)\n");
    printf("3. Solution B - Limit 4 Philosophers\n");
    printf("Masukkan pilihan (1/2/3): ");

```

```

    scanf("%d", &choice);
    printf("\n");

    if (choice == 1) {
        for (int i = 0; i < NUM_PHIL; i++)
            pthread_create(&philosopher[i], NULL, philosopher_deadlock, &phil_id[i]);
    } else if (choice == 2) {
        for (int i = 0; i < NUM_PHIL; i++)
            pthread_create(&philosopher[i], NULL, philosopher_ordering, &phil_id[i]);
    } else if (choice == 3) {
        for (int i = 0; i < NUM_PHIL; i++)
            pthread_create(&philosopher[i], NULL, philosopher_limit, &phil_id[i]);
    } else {
        printf("Pilihan tidak valid!\n");
        return 0;
    }

    for (int i = 0; i < NUM_PHIL; i++)
        pthread_join(philosopher[i], NULL);

    for (int i = 0; i < NUM_PHIL; i++)
        pthread_mutex_destroy(&chopstick[i]);
    sem_destroy(&room);

    return 0;
}

```

Cara Kompail dan run :

```
gcc -o dining_philosophers dining_philosophers.c
```

```
./dining_philosophers
```

Output :

```
🌸 Filsuf 0 sedang berpikir...
🌸 Filsuf 1 sedang berpikir...
🌸 Filsuf 2 sedang berpikir...
🌸 Filsuf 3 sedang berpikir...
🌸 Filsuf 4 sedang berpikir...
😋 Filsuf 0 lapar dan mencoba mengambil sumpit kiri 0
✅ Filsuf 0 memegang sumpit kiri 0
😋 Filsuf 2 lapar dan mencoba mengambil sumpit kiri 2
✅ Filsuf 2 memegang sumpit kiri 2
😋 Filsuf 3 lapar dan mencoba mengambil sumpit kiri 3
✅ Filsuf 3 memegang sumpit kiri 3
😋 Filsuf 4 lapar dan mencoba mengambil sumpit kiri 4
✅ Filsuf 4 memegang sumpit kiri 4
😋 Filsuf 1 lapar dan mencoba mengambil sumpit kiri 1
✅ Filsuf 1 memegang sumpit kiri 1
😋 Filsuf 0 mencoba mengambil sumpit kanan 1
😋 Filsuf 2 mencoba mengambil sumpit kanan 3
😋 Filsuf 3 mencoba mengambil sumpit kanan 4
😋 Filsuf 4 mencoba mengambil sumpit kanan 0
😋 Filsuf 1 mencoba mengambil sumpit kanan 2
```

3.2.2 Versi Resource Ordering

Filsuf dengan nomor genap mengambil sumpit kiri terlebih dahulu, sedangkan filsuf ganjil mengambil sumpit kanan terlebih dahulu. Teknik ini menghilangkan kondisi *circular wait* sehingga deadlock tidak terjadi.

Input :

```
GNU nano 7.2 dining_ordering.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>
#include <stdarg.h>
#include <stdlib.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philo[N];
int id_arr[N];
int eat_count[N];
struct timeval start_time;

void print_ts(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    struct timeval now;
    gettimeofday(&now, NULL);
    long elapsed = now.tv_sec - start_time.tv_sec;
    int mm = (elapsed / 60) % 60;
    int ss = elapsed % 60;
    printf("[%02d:%02d] ", mm, ss);
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    va_end(ap);
}

void* philosopher(void* arg) {
    int id = *(int*)arg;
    unsigned int seed = id + 1;
    while (1) {
        print_ts("🤔 Filsuf-%d sedang berpikir...", id+1);
        usleep((100 + (rand_r(&seed) % 900)) * 1000);
        print_ts("🍴 Filsuf-%d lapar, mencoba ambil sumpit...", id+1);
        int left = id;
        int right = (id + 1) % N;
        if ((id % 2) == 0) {
            print_ts(" // Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[left]);
            print_ts(" // Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[right]);
        } else {
            print_ts(" // Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[right]);
            print_ts(" // Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[left]);
        }
        print_ts("🍽️ Filsuf-%d sedang MAKAN...", id+1);
        eat_count[id]++;
        usleep((300 + (rand_r(&seed) % 700)) * 1000);
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        print_ts(" // Filsuf-%d menaruh kedua sumpit", id+1);
    }
    return NULL;
}

int main() {
    gettimeofday(&start_time, NULL);
    printf("=== DINING PHILOSOPHERS PROBLEM ===\n");

    printf("Solusi: Resource Ordering\n\n");
    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        id_arr[i] = i;
        eat_count[i] = 0;
    }
    for (int i = 0; i < N; i++) {
        pthread_create(&philo[i], NULL, philosopher, &id_arr[i]);
        usleep(100000);
    }
    sleep(60);
    for (int i = 0; i < N; i++) pthread_cancel(philo[i]);
    for (int i = 0; i < N; i++) pthread_join(philo[i], NULL);
    printf("\n=== STATISTIK (60 detik) ===\n");
    int total = 0;
    for (int i = 0; i < N; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    if (total > 0)
        printf("✅ Tidak ada deadlock!\n");
    else
        printf("⚠️ Kemungkinan deadlock atau starvation!\n");
    for (int i = 0; i < N; i++) pthread_mutex_destroy(&forks[i]);
    return 0;
}
```


Cara kompail dan run :

```
gcc -o dining_ordering dining_ordering.c
```

```
./dining_ordering
```

Output :

```
[00:00] 🤔 Filsuf-1 sedang berpikir...
[00:01] 🤔 Filsuf-2 sedang berpikir...
[00:01] 🤔 Filsuf-3 sedang berpikir...
[00:01] 🤔 Filsuf-4 sedang berpikir...
[00:01] 🍴 Filsuf-1 lapar, mencoba ambil sumpit...
[00:01] // Filsuf-1 mengambil sumpit kiri (0)
[00:01] // Filsuf-1 mengambil sumpit kanan (1)
[00:01] 🍽️ Filsuf-1 sedang MAKAN...
[00:01] 🤔 Filsuf-5 sedang berpikir...
[00:01] 🍴 Filsuf-4 lapar, mencoba ambil sumpit...
[00:01] // Filsuf-4 mengambil sumpit kanan (4)
[00:01] // Filsuf-4 mengambil sumpit kiri (3)
[00:01] 🍽️ Filsuf-4 sedang MAKAN...
[00:01] // Filsuf-1 menaruh kedua sumpit
[00:01] 🤔 Filsuf-1 sedang berpikir...
[00:01] 🍴 Filsuf-2 lapar, mencoba ambil sumpit...
[00:01] // Filsuf-2 mengambil sumpit kanan (2)
[00:01] // Filsuf-2 mengambil sumpit kiri (1)
[00:01] 🍽️ Filsuf-2 sedang MAKAN...
[00:02] 🍴 Filsuf-1 lapar, mencoba ambil sumpit...
[00:02] // Filsuf-1 mengambil sumpit kiri (0)
[00:02] // Filsuf-1 mengambil sumpit kanan (1)
[00:02] 🍴 Filsuf-3 lapar, mencoba ambil sumpit...
[00:02] // Filsuf-3 mengambil sumpit kiri (2)
[00:02] 🍴 Filsuf-5 lapar, mencoba ambil sumpit...
[00:02] // Filsuf-5 mengambil sumpit kiri (4)
[00:02] // Filsuf-4 menaruh kedua sumpit
[00:02] 🤔 Filsuf-4 sedang berpikir...
[00:02] // Filsuf-5 mengambil sumpit kanan (0)
[00:02] 🍴 Filsuf-4 lapar, mencoba ambil sumpit...
[00:02] // Filsuf-4 mengambil sumpit kanan (4)
[00:02] // Filsuf-2 menaruh kedua sumpit
[00:02] 🤔 Filsuf-2 sedang berpikir...
[00:02] 🍽️ Filsuf-1 sedang MAKAN...
[00:02] // Filsuf-3 mengambil sumpit kanan (3)
[00:02] 🍽️ Filsuf-3 sedang MAKAN...
```

3.2.3 Versi Limiting Philosophers

Menggunakan semaphore untuk membatasi maksimum 4 filsuf yang dapat mencoba mengambil sumpit secara bersamaan. Hal ini memastikan minimal satu filsuf dapat menyelesaikan makan tanpa menunggu sumpit.

Input :

```
GNU nano 7.2 dining_limiter.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <sys/time.h>
#include <stdarg.h>
#include <stdlib.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philo[N];
int id_arr[N];
int eat_count[N];
sem_t room;
struct timeval start_time;

void print_ts(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    struct timeval now;
    gettimeofday(&now, NULL);
    long elapsed = now.tv_sec - start_time.tv_sec;
    int mm = (elapsed / 60) % 60;
    int ss = elapsed % 60;
    printf("[%02d:%02d] ", mm, ss);
    vprintf(fmt, ap);
    printf("\n");
    fflush(stdout);
    va_end(ap);
}
```

```
void* philosopher(void* arg) {
    int id = *(int*)arg;
    unsigned int seed = id + 10;
    while (1) {
        print_ts("🤖 Filsuf-%d sedang berpikir...", id+1);
        usleep((100 + (rand_r(&seed) % 900)) * 1000);
        print_ts("🍴 Filsuf-%d lapar, mencoba masuk room...", id+1);
        sem_wait(&room);
        print_ts("🚪 Filsuf-%d masuk room (boleh mencoba ambil sumpit)", id+1);
        int left = id;
        int right = (id + 1) % N;
        if (left < right) {
            pthread_mutex_lock(&forks[left]);
            print_ts("🍴 Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
            pthread_mutex_lock(&forks[right]);
            print_ts("🍴 Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
        } else {
            pthread_mutex_lock(&forks[right]);
            print_ts("🍴 Filsuf-%d mengambil sumpit kanan (%d)", id+1, right);
            pthread_mutex_lock(&forks[left]);
            print_ts("🍴 Filsuf-%d mengambil sumpit kiri (%d)", id+1, left);
        }
        print_ts("🍽️ Filsuf-%d sedang MAKAN...", id+1);
        eat_count[id]++;
        usleep((300 + (rand_r(&seed) % 700)) * 1000);
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        print_ts("🍴 Filsuf-%d menaruh kedua sumpit", id+1);
        sem_post(&room);
        print_ts("🚪 Filsuf-%d keluar room", id+1);
    }
    return NULL;
}
```

Cara kompail dan run :

```
gcc -o dining_limiter dining_limiter.c
```

```
./dining_limiter
```

Ouput :

```
[00:01] 🍴 Filsuf-5 masuk room (boleh mencoba ambil sumpit)
[00:02] // Filsuf-4 menaruh kedua sumpit
[00:02] 🚶 Filsuf-4 keluar room
[00:02] 🤔 Filsuf-4 sedang berpikir...
[00:02] // Filsuf-2 menaruh kedua sumpit
[00:02] 🚶 Filsuf-2 keluar room
[00:02] 🤔 Filsuf-2 sedang berpikir...
[00:02] // Filsuf-1 mengambil sumpit kanan (1)
[00:02] 🍽️ Filsuf-1 sedang MAKAN...
[00:02] 🍴 Filsuf-2 lapar, mencoba masuk room...
[00:02] 🍴 Filsuf-2 masuk room (boleh mencoba ambil sumpit)
[00:02] 🍴 Filsuf-3 lapar, mencoba masuk room...
[00:02] 🍴 Filsuf-3 masuk room (boleh mencoba ambil sumpit)
[00:02] // Filsuf-3 mengambil sumpit kiri (2)
[00:02] // Filsuf-3 mengambil sumpit kanan (3)
[00:02] 🍽️ Filsuf-3 sedang MAKAN...
[00:02] 🍴 Filsuf-4 lapar, mencoba masuk room...
[00:02] // Filsuf-3 menaruh kedua sumpit
[00:02] 🚶 Filsuf-3 keluar room
[00:02] 🤔 Filsuf-3 sedang berpikir...
[00:02] 🍴 Filsuf-4 masuk room (boleh mencoba ambil sumpit)
[00:02] // Filsuf-4 mengambil sumpit kiri (3)
[00:02] // Filsuf-4 mengambil sumpit kanan (4)
[00:02] 🍽️ Filsuf-4 sedang MAKAN...
[00:02] // Filsuf-1 menaruh kedua sumpit
[00:02] 🚶 Filsuf-1 keluar room
[00:02] 🤔 Filsuf-1 sedang berpikir...
[00:02] // Filsuf-2 mengambil sumpit kiri (1)
[00:02] // Filsuf-2 mengambil sumpit kanan (2)
[00:02] 🍽️ Filsuf-2 sedang MAKAN...
[00:02] // Filsuf-5 mengambil sumpit kanan (0)
[00:03] 🍴 Filsuf-3 lapar, mencoba masuk room...
[00:03] 🍴 Filsuf-3 masuk room (boleh mencoba ambil sumpit)
[00:03] 🍴 Filsuf-1 lapar, mencoba masuk room...
[00:03] // Filsuf-2 menaruh kedua sumpit
[00:03] 🚶 Filsuf-2 keluar room
```

4. Tabel Perbandingan Efisiensi dan Fairness

Pendekatan / Solusi	Efisiensi (Pemakaian Fork & CPU)	Fairness (Keadilan Akses)	Deadlock	Starvation
Tanpa Pencegahan (Deadlock terjadi)	Rendah (semua menunggu)	Tidak adil	Ya	Ya
Menggunakan batas maksimum (4 dari 5 boleh makan)	Tinggi (hindari menunggu semua)	Cukup adil	Tidak	Sangat kecil
Menggunakan even-odd (par genap/ganjil ambil berbeda)	Sedang	Cukup adil	Tidak	Bisa terjadi jika urutan tertentu
Menggunakan waiter / semaphore global	Sangat efisien (terkontrol)	Paling adil (berurutan)	Tidak	Tidak

5. Analisis Mendalam

a. Mengapa deadlock bisa terjadi?

Deadlock terjadi karena **semua filosof memegang satu garpu dan menunggu garpu yang lain**, yang sedang dipegang oleh tetangganya.

Empat kondisi deadlock terpenuhi:

1. **Mutual exclusion** – garpu hanya bisa dipegang satu filosof.
2. **Hold and wait** – tiap filosof memegang satu garpu dan menunggu garpu lain.
3. **No preemption** – garpu tidak bisa direbut.
4. **Circular wait** – setiap filosof menunggu garpu dari filosof di sebelahnya.

Contoh: semua filosof mengambil garpu kiri bersamaan → semuanya menunggu garpu kanan → tidak ada yang bisa lanjut makan → *deadlock*.

6. Bagaimana masing-masing solusi bekerja?

- **Tanpa solusi (demonstrasi deadlock):**
Semua mengambil garpu kiri, lalu menunggu garpu kanan → terjadi deadlock.

- **Solusi 1 – Limiting Philosophers (batas maksimal 4):**
Hanya 4 dari 5 filosof yang boleh mencoba makan sekaligus.
Karena minimal 1 filosof tidak mengambil garpu, siklus menunggu terputus → *deadlock* tidak terjadi.
- **Solusi 2 – Even-Odd Rule (aturan ganjil-genap):**
Filosof ganjil mengambil garpu kiri dulu, genap mengambil garpu kanan dulu.
Pola ini memecah siklus tunggu → mencegah *deadlock* sebagian besar waktu.
- **Solusi 3 – Waiter (semaphore global):**
Ada satu *waiter* (semaphore) yang mengatur izin makan. Filosof hanya boleh makan jika dua garpu tersedia.
Ini mencegah *deadlock* total karena keputusan terpusat.

7. Solusi mana yang paling adil (fair)?

- **Solusi dengan waiter / semaphore global** adalah yang paling adil.
Karena setiap filosof akan mendapatkan giliran makan secara bergantian (tergantung ketersediaan garpu), bukan berdasarkan urutan thread atau keberuntungan.
- *Limiting philosopher* cukup adil tapi masih mungkin beberapa filosof sering menunggu lebih lama.

8. Apakah ada kemungkinan starvation?

- **Tanpa solusi:** sangat mungkin → karena ada filosof yang terus menunggu selamanya.
- **Limiting philosophers:** kecil kemungkinan starvation, tetapi masih bisa jika scheduler tidak adil.
- **Even-Odd:** masih mungkin starvation karena urutan akses tidak selalu merata.
- **Waiter / semaphore global:** tidak ada starvation, karena waiter menjamin giliran semua filosof secara bergantian.

KESIMPULAN DAN SARAN

4.1 Kesimpulan

1. Deadlock terjadi jika semua syarat *Coffman* terpenuhi.
2. Pada *Dining Philosophers Problem*, deadlock dapat dicegah dengan:
 - Mengubah urutan pengambilan sumber daya (*Resource Ordering*).
 - Membatasi jumlah proses yang dapat mengakses sumber daya (*Limiting Philosophers*).
3. Sinkronisasi dengan mutex dan semaphore sangat penting dalam pemrograman multithread untuk menjaga konsistensi data dan mencegah kebuntuan (*deadlock*).

4.2 Saran

- Gunakan pendekatan semaphore untuk sistem dengan jumlah thread banyak agar lebih adil.
- Tambahkan *timeout* agar proses yang terlalu lama menunggu dapat dibatalkan secara otomatis.
- Kembangkan sistem monitoring untuk mendeteksi *deadlock* lebih cepat.

DAFTAR PUSTAKA

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Tanenbaum, A. S. (2015). *Modern Operating Systems* (4th ed.). Pearson.
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles*.
4. Materi Praktikum Sistem Operasi – Universitas XYZ (2025).